



SERVICIO NACIONAL DE APRENDIZAJE · SENA

Camila nails

Diseño y desarrollo de servicios web - caso
GA7-220501096-AA5-EV01

Aprendiz:	Andrés Mauricio Valencia Arango
Programa:	Tecnología en Análisis y Desarrollo de Software
Ficha:	3186645
Instructor:	Adonay Sánchez

Índice

Contenido	Pág ina
1. Introducción	3
2. Objetivos	4
3. Descripción y desarrollo del proyecto	5
4. Tecnologías y arquitectura	7
5. Evidencias visuales del proyecto	8
6. Conclusiones	11

1. Introducción

El presente documento describe el diseño y desarrollo de **Camila nails**, un servicio web creado para presentar los servicios de un estudio de uñas, orientar a sus clientes y facilitar la solicitud de citas. El proyecto responde a la necesidad de contar con una presencia digital clara, atractiva y funcional, donde la información comercial pueda consultarse desde distintos dispositivos.

La solución fue construida como una aplicación web moderna con React y Vite. Su interfaz incorpora una identidad visual cálida, navegación por secciones, catálogo de estilos, información sobre esmaltes, guía de enfermedades de las uñas, autenticación de usuarios, perfil personal y un flujo de agendamiento que conecta con WhatsApp.

También se implementó una separación de permisos: los clientes consultan el contenido y gestionan su perfil, mientras que el administrador cuenta con herramientas privadas para autorizar cambios y administrar imágenes del catálogo.

2. Objetivos

2.1 Objetivo general

Diseñar y desarrollar un servicio web para Camila nails que permita presentar sus servicios de manera atractiva, facilitar la interacción con los clientes y ofrecer herramientas de administración de contenidos.

2.2 Objetivos específicos

- Construir una interfaz responsive, clara y coherente con la identidad visual del emprendimiento.
- Organizar la información en las secciones de historia, estilos, esmaltes y cuidado de las uñas.
- Implementar registro e inicio de sesión para controlar el acceso a funciones privadas.
- Permitir que clientes y administrador actualicen el nombre de su perfil.
- Restringir las herramientas de edición, aprobación y carga de imágenes al administrador.
- Facilitar la solicitud de citas mediante un formulario conectado a WhatsApp.

3. Descripción y desarrollo del proyecto

3.1 Interfaz pública

La página de inicio presenta la propuesta de valor del estudio, un bloque visual de inspiración, servicios destacados y llamados a la acción. La navegación permite acceder a la historia profesional, estilos de uñas, marcas de esmaltes y guía de enfermedades.

3.2 Autenticación y perfiles

El sistema cuenta con registro e inicio de sesión. La sesión conserva los datos básicos del usuario y distingue entre el rol de cliente y el rol de administrador. Cada usuario autenticado dispone de un perfil desde el cual puede cambiar su nombre sin modificar su usuario, correo ni permisos.

3.3 Agendamiento

La sección de contacto está protegida para usuarios autenticados. El formulario solicita nombre, teléfono, fecha, hora y notas; luego prepara un mensaje codificado y abre WhatsApp con la información de la reserva.

3.4 Administración

El administrador puede acceder al panel privado, revisar propuestas, aprobar o rechazar cambios y subir imágenes para los servicios. Estas opciones no se muestran a los clientes, conservando una experiencia pública limpia y evitando exponer controles de gestión.

4. Tecnologías y arquitectura

Frontend	React 19, React Router y Vite.
Estilos	CSS con variables, diseño responsive, tarjetas, navegación tipo burbuja y tipografías editoriales.
Backend	Node.js con Express y endpoints REST para autenticación, perfiles, propuestas y archivos.
Persistencia	LowDB con archivo JSON para usuarios, propuestas e imágenes; localStorage como respaldo offline.
Seguridad funcional	Rutas protegidas, rol administrativo, contraseñas almacenadas mediante hash y respuestas sin contraseña.
Comunicación	Proxy de Vite hacia el servidor local y enlace de reserva mediante WhatsApp.

4.1 Estructura principal

src/	
App.jsx	Rutas y composición principal
AuthProvider.jsx	Estado de autenticación y perfil
components/	Barra, lightbox y rutas protegidas
pages/	Inicio, catálogo, contacto, login y administración
assets/	Logo, fotografías y recursos visuales
server/	
index.js	API Express
db.json	Persistencia local

4.2 Proceso de construcción

El servicio se construyó por etapas. Primero se organizó la aplicación React y sus rutas; luego se incorporaron los componentes visuales, la autenticación, el perfil de usuario y finalmente el backend para persistir la información.

Etapas 1 · Estructura de la aplicación

src/App.jsx

```
01 // Componente raíz: conecta autenticación, navegación y todas las páginas.
02 import './App.css'
03 import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
04 import { AuthProvider } from './AuthProvider'
05 import Navbar from './components/Navbar'
06 import Home from './pages/Home'
07 import About from './pages/About'
08 import Styles from './pages/Styles'
09 import Polishes from './pages/Polishes'
10 import Diseases from './pages/Diseases'
11 import Contact from './pages/Contact'
12 import Login from './pages/Login'
13 import Register from './pages/Register'
14 import Profile from './pages/Profile'
15 import AdminPanel from './pages/AdminPanel'
16 import ProtectedRoute from './components/ProtectedRoute'
17
18 function App() {
19   // El proveedor mantiene la sesión disponible para toda la aplicación.
20   return (
21     <AuthProvider>
22       <BrowserRouter>
23         <Navbar />
24         <main className="app-container">
25           <Routes>
```

Las rutas separan las páginas públicas de las secciones protegidas.

Etapas 2 · Gestión de usuarios

src/AuthProvider.jsx

```

01 import React, { createContext, useEffect, useState } from 'react'
02
03 export const AuthContext = createContext(null)
04
05 export function AuthProvider({ children }) {
06   // Estado del usuario autenticado (sin contraseña)
07   const [user, setUser] = useState(null)
08
09   useEffect(() => {
10     const stored = localStorage.getItem('kb_user')
11     if (stored) setUser(JSON.parse(stored))
12   }, [])
13
14   // Helper: hash SHA-256 para contraseñas (cliente). Devuelve hex string.
15   async function hashPassword(password) {
16     const enc = new TextEncoder()
17     const data = enc.encode(password)
18     const hashBuffer = await crypto.subtle.digest('SHA-256', data)
19     const hashArray = Array.from(new Uint8Array(hashBuffer))
20     return hashArray.map((b) => b.toString(16).padStart(2, '0')).join('')
21   }
22
23   /**
24    * register({ name, username, email, password })
25    * - Registra usuario en `kb_users` con password hash.
26    * - Devuelve { success, message/error } y hace login automático.
27    */
28   async function register({ name, username, email, password }) {
29     if (!username || !password || !email) {
30       return { success: false, error: 'Faltan campos obligatorios' }
31     }
32     // Try server-side registration first

```

El proveedor centraliza login, registro, roles y actualización del perfil.

Etapas 3 · Servicios del backend

server/index.js

```
01 const express = require('express')
02 const cors = require('cors')
03 const path = require('path')
04 const fs = require('fs')
05 const multer = require('multer')
06 const bcrypt = require('bcryptjs')
07 const { Low } = require('lowdb')
08 const { JSONFile } = require('lowdb/node')
09
10 const PORT = process.env.PORT || 4000
11 const app = express()
12
13 app.use(cors())
14 app.use(express.json())
15
16 // Ensure uploads folder exists
17 const uploadsDir = path.join(__dirname, 'uploads')
18 if (!fs.existsSync(uploadsDir)) fs.mkdirSync(uploadsDir, { recursive: true })
19
20 // Multer setup
21 const storage = multer.diskStorage({
22   destination: function (req, file, cb) { cb(null, uploadsDir) },
23   filename: function (req, file, cb) {
24     const unique = Date.now() + '-' + Math.round(Math.random() * 1e9)
25     const name = unique + path.extname(file.originalname)
26     cb(null, name)
27   }
28 })
29 const upload = multer({ storage })
30
31 // Initialize lowdb (JSON file storage)
32 const dbFile = path.join(__dirname, 'db.json')
33 const adapter = new JSONFile(dbFile)
34 const db = new Low(adapter, { users: [], pending: [], uploads: [] })
35
```

La API Express recibe las solicitudes de registro, login y actualización del nombre.

Etapas 4 · Resultado visual

El resultado combina la lógica anterior con una identidad visual de tonos cálidos, navegación tipo burbuja, catálogo de servicios e imágenes de referencia.

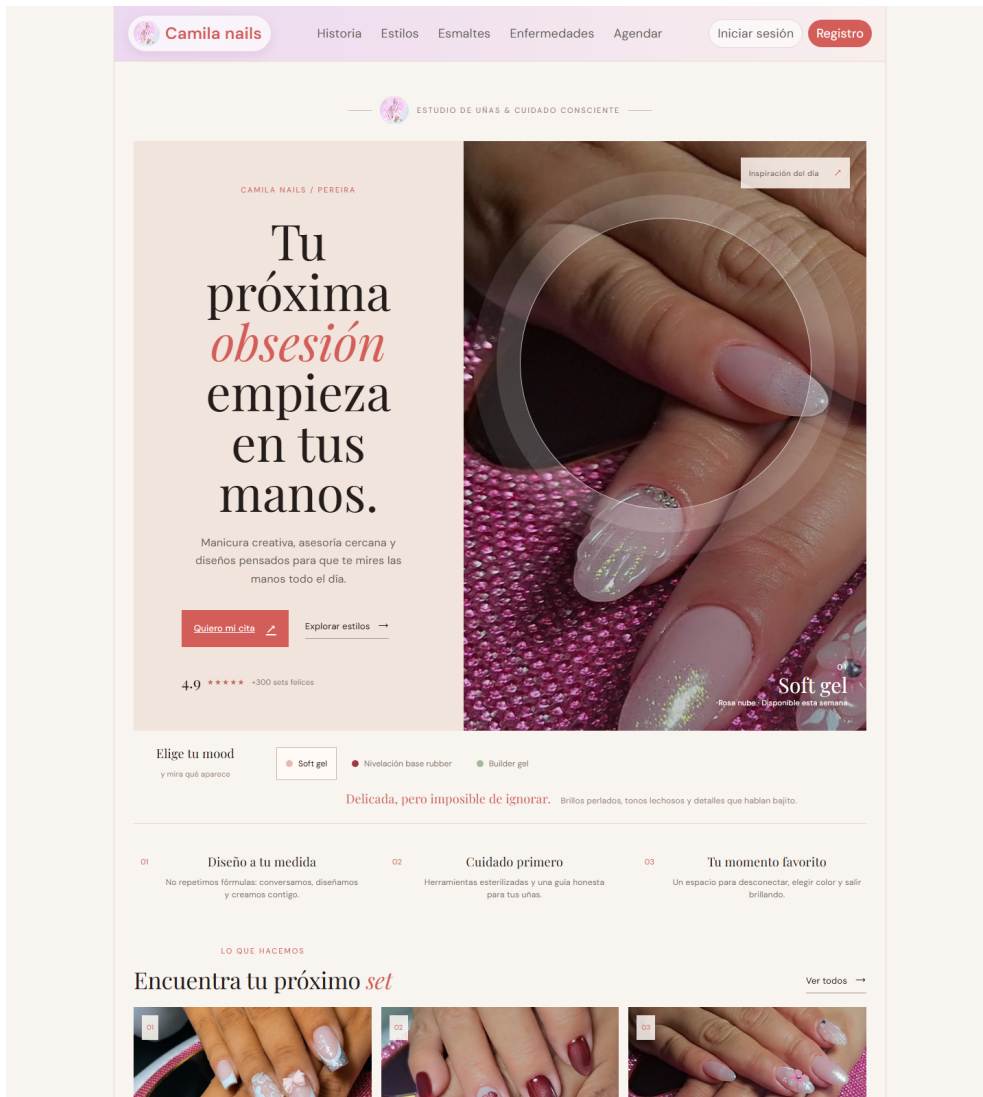


Figura 5. Captura real de la página principal en funcionamiento.

5. Impresiones visuales del proyecto

Las siguientes impresiones documentan la interfaz visual implementada, sus recursos gráficos y las principales pantallas del servicio web.



Figura 1. Identidad visual de Camila nails usada en la página de inicio.



Figura 2. Recursos visuales del catálogo de estilos y servicios de uñas.

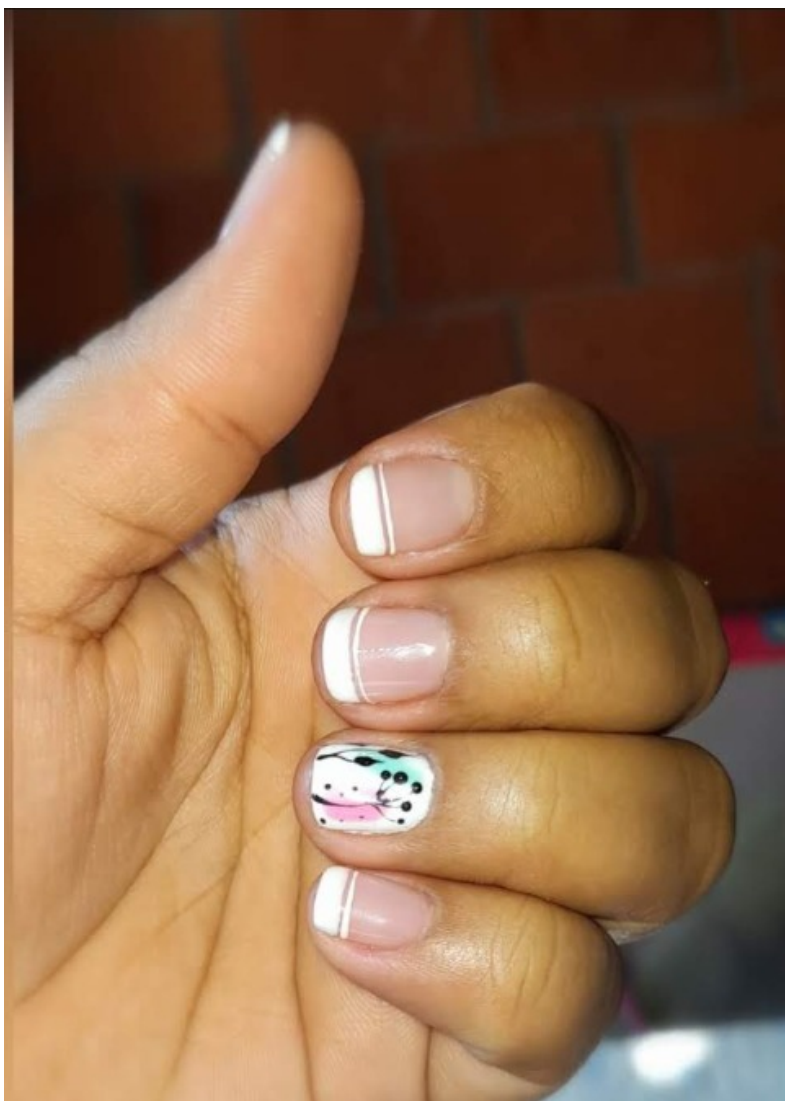


Figura 3. Identidad visual de la sección de marcas de esmaltes.



Figura 4. Logo utilizado en las pantallas de inicio de sesión y registro.

6. Conclusiones

El desarrollo de Camila nails permitió integrar diseño visual, navegación, autenticación y comunicación con el cliente en un mismo servicio web. La interfaz presenta la marca de forma atractiva y organiza la información para que pueda consultarse con facilidad.

La incorporación de roles mejora la separación entre la experiencia del cliente y las tareas administrativas. De esta manera, los visitantes encuentran una página limpia y los usuarios autenticados pueden acceder a funciones específicas como el agendamiento y la edición de su perfil.

Finalmente, el proyecto demuestra la aplicación práctica de tecnologías actuales para construir una solución escalable. Como trabajo futuro se podrían incorporar autenticación con tokens, una base de datos de producción, confirmaciones automáticas de citas y un editor visual para contenidos.

Documento elaborado como evidencia del proyecto "Diseño y desarrollo de servicios web - caso. GA7-220501096-AA5-EV01".